

SMART BUILDING ENERGY CONSUMPTION PREDICTION

Using Machine Learning

Sneha Prathipati

Roll No: N210970

Under the Guidance of:

P. Suresh Sir

Department of Electronics and Communication Engineering
RAJIV GANDHI UNIVERSITY OF KNOWLEDGE TECHNOLOGIES
Academic Year 2025–2026

Contents

- 1 Abstract
- 2 Introduction
- 3 System Architecture
- 4 Workflow
- 5 Working Principle
- 6 Model Theory
- 7 App Demo

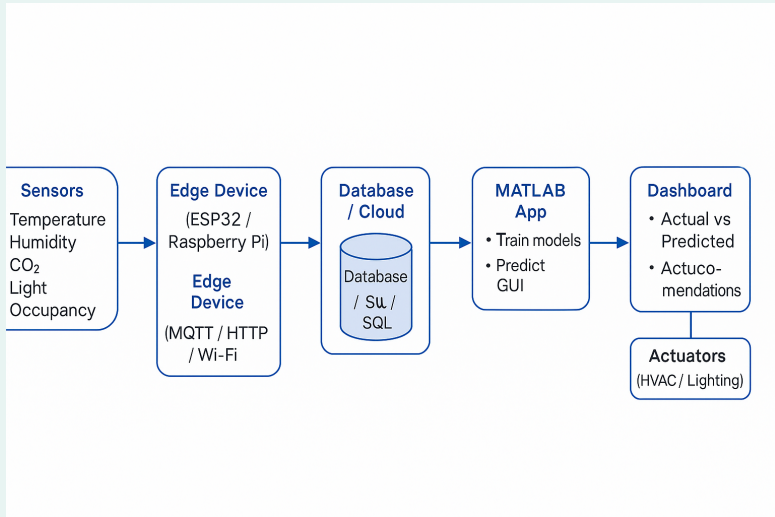
Abstract

- A MATLAB-based prototype predicts short-horizon building energy consumption using Temperature, Humidity, CO₂, Light and Occupancy.
- The system includes preprocessing, model training, evaluation, feature-importance analysis, and a GUI with prediction + recommendations.
- This presentation explains methodology, working principle, model theory, results and the app workflow.

Introduction

- Buildings consume high energy — mainly HVAC and lighting.
- Predictive analytics reduces cost and improves comfort.
- This project is a MATLAB-based end-to-end energy prediction prototype, scalable to IoT systems.

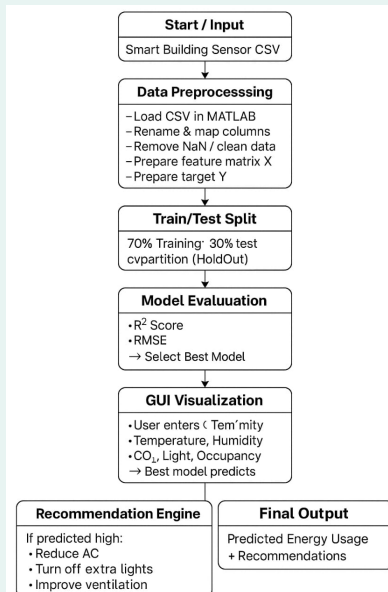
System Architecture



Project Workflow — Steps

- 1 Load data (CSV).
- 2 Preprocess (clean, map, remove NaN, build X and Y).
- 3 Split (70/30 holdout).
- 4 Train models (Linear, Tree, RF, SVR, Boosted).
- 5 Evaluate (R^2 , RMSE, MAE).
- 6 Predict & Recommend in the App.

Workflow Diagram — Flowchart



Working Principle

- Sensors measure: Temp, Humidity, CO₂, Light, Occupancy.
- MATLAB cleans and prepares data.
- Multiple models are trained.
- Best model integrated into GUI.
- GUI outputs prediction + energy-saving suggestions.

Linear Regression

$$\hat{y} = \beta_0 + \sum_i \beta_i x_i$$

- Predicts the target as a weighted sum of input features.
- Very easy to explain: each β_i shows how much x_i changes the output.
- Works well when relationship is roughly straight-line (linear).
- Fast to train and good as a baseline model.

Terms: $\hat{y}$ = predicted value, β_i = weight (coefficient), x_i = input feature.

Decision Tree (Intuitive Rules)

Prediction = mean(y in a leaf)

- Splits data by rules (if-then) on feature values (like a flowchart).
- Easy to read: you can trace why a decision was made.
- Can capture nonlinear patterns without scaling features.
- Risk: can overfit (memorise noise) if the tree grows too deep.

Terms: Node = a split point, Leaf = final node with predicted value, Depth = tree height (complexity).

Random Forest (Many Trees)

$$\hat{y}(x) = \frac{1}{T} \sum_{t=1}^T h_t(x)$$

- Builds many decision trees on random subsets of data/features.
- Final prediction is the average of all trees (reduces overfitting).
- More stable and accurate than one tree for most tabular data.
- Good default for prediction tasks with mixed features.

Terms: T = number of trees, $h_t(x)$ = prediction from tree t , bagging = sampling technique.

Support Vector Regression (SVR)

$$\min \frac{1}{2} \|w\|^2 + C \sum (\xi_i + \xi_i^*)$$

- Tries to fit a function within an “epsilon-tube” around data (ignores small errors).
- Uses kernels to capture nonlinear relationships (e.g., RBF kernel).
- Sensitive to feature scale → standardize inputs before training.
- Works well with medium-sized datasets and careful tuning.

Terms: w = model weights, C = penalty for errors, ξ = slack variables (errors beyond epsilon).

Gradient Boosted Trees (Boosting)

$$F_m(x) = F_{m-1}(x) + \gamma_m h_m(x)$$

- Builds trees sequentially; each new tree corrects previous errors (learns residuals).
- Often yields very high accuracy on tabular data when tuned well.
- More training time and careful tuning (learning rate, tree depth) needed.
- Combine many weak learners into a strong learner.

Terms: F_m = ensemble after m trees, h_m = new tree, γ_m = step size (learning rate).

R² and Model Performance

R² Score:

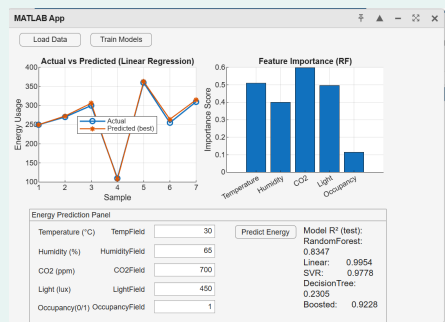
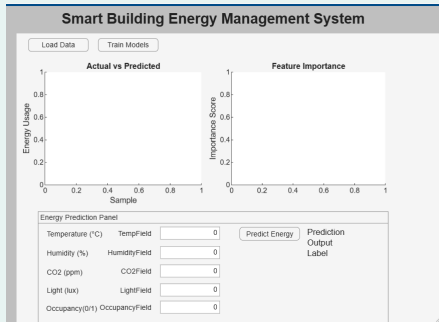
The coefficient of determination (R²) measures how well a model explains the variance in the target variable. A value closer to 1 indicates better predictive performance.

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

Model Performance

Model	R ²
Random Forest	0.85
Boosted Trees	0.82
SVR	0.77
Linear Reg.	0.70
Decision Tree	0.68

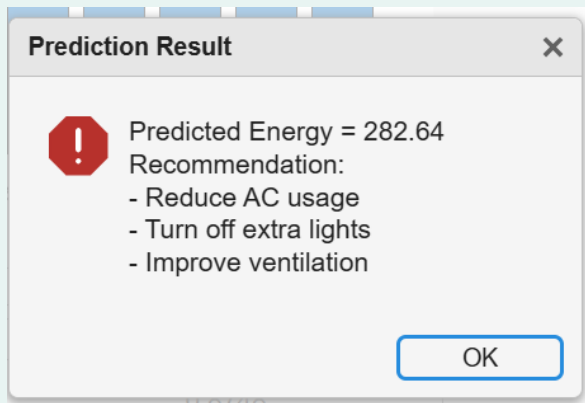
App — Main Screens



App — Input Panel

Energy Prediction Panel			
Temperature (°C)	TempField	24	Predict Energy Model R² (test): RandomForest: 0.9372 Linear: 0.9978 SVR: 0.9816 DecisionTree: 0.5514 Boosted: 0.8896
Humidity (%)	HumidityField	45	
CO2 (ppm)	CO2Field	550	
Light (lux)	LightField	350	
Occupancy(0/1)	OccupancyField	1	

App — Prediction Popup



Outputs

- Predicted energy usage.
- Feature importance.
- Model performance metrics.
- Recommendations.

Demo Steps

- ① Load CSV.
- ② Train models.
- ③ Enter live inputs.
- ④ Predict energy.
- ⑤ Review recommendations.

Example — Live Prediction

Input: 27°C, 60%, 800 ppm, 400 lux, Occupancy 1

Prediction: **245 units**

Actions: reduce AC, dim lights, increase ventilation.

Conclusion

- Complete MATLAB prototype for building energy prediction.
- Ensemble models give strong accuracy.
- GUI enables practical use + recommendations.

Thank you

Thank you!